

Otherwise:

Welcome back, Eric!

This is the output you see if the program was already run at least once. Even though the data in this section is just a single string, the program would work just as well with any data that can be converted to a JSON-formatted string.

Refactoring

Often, you'll come to a point where your code will work, but you'll recognize that you could improve the code by breaking it up into a series of functions that have specific jobs. This process is called *refactoring*. Refactoring makes your code cleaner, easier to understand, and easier to extend.

We can refactor *remember_me.py* by moving the bulk of its logic into one or more functions. The focus of *remember_me.py* is on greeting the user, so let's move all of our existing code into a function called `greet_user()`:

```
remember_me.py from pathlib import Path
import json

def greet_user():
    ❶ """Greet the user by name."""
    path = Path('username.json')
    if path.exists():
        contents = path.read_text()
        username = json.loads(contents)
        print(f>Welcome back, {username}!")
    else:
        username = input("What is your name? ")
        contents = json.dumps(username)
        path.write_text(contents)
        print(f>We'll remember you when you come back, {username}!")

greet_user()
```

Because we're using a function now, we rewrite the comments as a docstring that reflects how the program currently works ❶. This file is a little cleaner, but the function `greet_user()` is doing more than just greeting the user—it's also retrieving a stored username if one exists and prompting for a new username if one doesn't.

Let's refactor `greet_user()` so it's not doing so many different tasks. We'll start by moving the code for retrieving a stored username to a separate function:

```
from pathlib import Path
import json

def get_stored_username(path):
    ❶ """Get stored username if available."""
```
